

NAME:Shrutha

ROLL NO.: 241801265

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.

2. The next line contains m integers representing the sizes of each memory block.

3. The next line contains an integer n representing the number of processes.

4. The next line contains n integers representing the size of each process.

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int m, n;
5
6     printf("Enter number of memory blocks:\n");
7     scanf("%d", &m);
8
9     int block[m], original[m];
10
11     printf("Enter size of each block:\n");
12     for(int i = 0; i < m; i++) {
13         scanf("%d", &block[i]);
14         original[i] = block[i];
15     }
16
17     printf("Enter number of processes:\n");
18     scanf("%d", &n);
19
20     int process[n];
21
22     printf("Enter size of each process:\n");
23     for(int i = 0; i < n; i++) {
24         scanf("%d", &process[i]);
```

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.

2. The next line contains m integers representing the sizes of each memory block.

3. The next line contains an integer n representing the number of processes.

4. The next line contains n integers representing the sizes of each process.

Select Language: C (GCC 9.2.0)

```
25 }
26
27 for(int i = 0; i < n; i++) {
28     int flag = 0;
29
30     for(int j = 0; j < m; j++) {
31         if(block[j] >= process[i]) {
32
33             int fragment = block[j] - process[i];
34
35             printf("Process %d of size %d is allocated to block %d of size %d with fragment %d\n",
36                 i + 1, process[i], j + 1, original[j], fragment);
37
38             block[j] -= process[i];
39             flag = 1;
40             break;
41         }
42     }
43
44     if(flag == 0) {
45         printf("Process %d of size %d is not allocated",
46             i + 1, process[i]); // * NO \n here
47     }
48 }
```

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500
with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600
with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500
with Fragment 176
Process 4 of size 426 is not allocated
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated
```